

ui.header 全面详细解析

`ui.header` 是 NiceGUI 框架中用于构建页面顶部导航栏 / 标题栏的核心布局组件，基于 Quasar 框架的 Header 组件封装，提供了固定定位、阴影效果、响应式适配等基础能力，同时支持样式自定义、事件绑定、数据绑定等高级功能，是构建结构化页面布局的关键元素之一。以下从核心特性、初始化参数、属性与方法、使用示例、注意事项等方面进行全面解析。

一、核心特性

- 布局定位**：默认固定在页面顶部（`fixed=True`），滚动页面时保持可见，也可配置为随内容滚动。
- 视觉定制**：支持边框显示、阴影效果、内容换行等视觉配置，可通过 `style`、`classes` 灵活调整样式。
- 内容适配**：默认自动添加滚动内边距（`add_scroll_padding=True`），避免锚点链接目标被 header 遮挡。
- 功能扩展**：支持事件监听、数据绑定、动态资源添加等，可与其他 NiceGUI 组件深度集成（如按钮、菜单、输入框等）。
- 继承特性**：继承自 `ValueElement`、`Element`、`Visibility` 基类，拥有通用的元素操作能力（如显示 / 隐藏、删除、更新等）。

二、初始化参数 (Initializer)

`ui.header` 的初始化参数用于配置组件基础行为和样式，所有参数均为可选，默认值已优化适配常见场景：

参数名	类型	说明	默认值
value	bool	是否默认展开 header（header 通常为常驻组件，此参数控制初始显示状态）	True
fixed	bool	是否固定在页面顶部：- True：滚动时保持在顶部，不随内容滚动- False：随页面内容一起滚动	True
bordered	bool	是否显示底部边框，用于区分 header 与页面主体内容	False
elevated	bool	是否添加阴影效果，增强视觉层次感	False
wrap	bool	当内容超出 header 宽度时，是否允许内容换行显示	True
add_scroll_padding	bool	是否自动为页面添加顶部内边距，避免锚点链接（如 <code>#section1</code> ）的目标元素被固定定位的 header 遮挡	True

三、核心属性 (Properties)

`ui.header` 继承了基类的核心属性，支持动态获取和设置组件状态：

属性名	类型	说明
classes	Classes[Self]	组件的 HTML 类名，用于通过 Tailwind/Quasar 类调整样式（支持链式调用修改）

属性名	类型	说明
client	Client	组件所属的客户端实例，用于关联 websocket 连接、布局上下文等
html_id	str	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，可用于直接操作 DOM）
is_deleted	bool	组件是否已被删除（只读）
is_ignoring_events	bool	组件是否正在忽略事件（只读）
parent_slot	Slot None	组件的父插槽（可设置，用于调整组件在父容器中的插槽位置）
props	Props[Self]	组件的 Quasar props，用于扩展组件原生能力（支持链式调用添加 / 删除）
style	Style[Self]	组件的内联 CSS 样式（支持链式调用修改，如 <code>style('background-color: #fff')</code> ）
value	BindableProperty	组件的核心状态值（与初始化 <code>value</code> 参数对应，可绑定数据动态控制显示状态）
visible	BindableProperty	组件的可见性（可绑定数据动态控制显示 / 隐藏）

四、关键方法（Methods）

`ui.header` 提供了丰富的方法用于组件操作、事件绑定、数据绑定等，以下是常用核心方法分类解析：

1. 基础操作方法

方法名	参数	说明
show()	-	显示 header 组件
hide()	-	隐藏 header 组件
toggle()	-	切换 header 的显示 / 隐藏状态
delete()	-	删除 header 组件及所有子元素
clear()	-	清除 header 中的所有子元素（保留 header 本身）
update()	-	同步组件状态到客户端，修改样式 / 属性后需调用以生效

2. 样式与属性修改方法

方法名	核心参数	说明
default_classes()	add/remove/toggle/replace	批量修改组件默认类名（需在组件实例化前调用，作用于所有同类组件）
default_props()	add/remove	批量修改组件默认 Quasar props（需在实例化前调用）

方法名	核心参数	说明
default_style()	add/remove/replace	批量修改组件默认 CSS 样式（需在实例化前调用）
style()	css_str	设置组件内联样式（支持链式调用，如 <code>ui.header().style('height: 60px').style('line-height: 60px')</code> ）
classes()	class_str	设置组件 HTML 类名（如 <code>classes('flex justify-between items-center')</code> ）
props()	props_str	设置 Quasar 原生属性（如 <code>props('flat bordered')</code> ）

3. 数据绑定方法

用于将 header 的状态（`value/visible`）与 Python 对象属性动态绑定，支持单向 / 双向同步：

方法名	核心参数	说明
bind_value()	target_object, target_name	双向绑定：header 的 <code>value</code> 与目标对象的属性同步（一方修改，另一方自动更新）
bind_value_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新 header 的 <code>value</code>
bind_value_to()	target_object, target_name	单向绑定（从组件到目标）：header 的 <code>value</code> 变化时，同步更新目标对象属性
bind_visibility()	target_object, target_name	双向绑定：header 的可见性与目标对象属性同步
bind_visibility_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新 header 可见性
bind_visibility_to()	target_object, target_name	单向绑定（从组件到目标）：header 可见性变化时，同步更新目标对象属性

4. 事件绑定方法

方法名	核心参数	说明
on()	type, handler, js_handler	绑定组件事件（如 <code>click</code> 、 <code>mousedown</code> 等），支持 Python 处理器或 JavaScript 处理器
on_value_change()	callback	绑定 <code>value</code> 变化事件：当 header 的 <code>value</code> 状态改变时触发回调函数

5. 其他实用方法

方法名	核心参数	说明
tooltip()	text	为 header 添加 tooltip 提示（鼠标悬浮时显示）
mark()	*markers	为组件添加标记，用于测试查询或依赖管理
move()	target_container, target_index	将 header 移动到其他父容器中
add_resource()	path	为 header 添加静态资源（如 CSS/JS 文件）
add_dynamic_resource()	name, function	为 header 添加动态资源（通过函数返回资源响应）

五、使用示例

1. 基础用法：简单顶部导航栏

```
from nicegui import ui

@ui.page('/basic_header')
def basic_header_demo():
    # 创建带阴影、边框的固定顶部 header
    with ui.header(elevated=True, bordered=True).style('background-color: #3874c8; color: white'):
        # 左侧标题
        ui.label('我的应用').classes('text-xl font-bold')
        # 右侧按钮（通过 classes 实现右对齐）
        with ui.row().classes('m1-auto'):
            ui.button('首页', on_click=lambda: ui.notify('点击首页')).props('flat color=white')
            ui.button('设置', on_click=lambda: ui.notify('点击设置')).props('flat color=white')
            ui.button('退出', on_click=lambda: ui.notify('点击退出')).props('flat color=white')

    # 页面主体内容（100行文本用于测试滚动）
    [ui.label(f'页面内容第 {i} 行') for i in range(100)]

ui.run()
```

2. 高级用法：动态绑定与事件监听

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.header_visible = True # 控制 header 可见性的状态变量
```

```

@ui.page('/advanced_header')
def advanced_header_demo():
    state = AppState()

    # 创建 header 并绑定可见性到 state.header_visible
    header = ui.header(
        fixed=True,
        elevated=True,
        style='background-color: #f5f5f5'
    ).bind_visibility_from(state, 'header_visible')

    with header:
        ui.label('动态 Header 演示').classes('text-lg')
        # 右侧控制按钮
        with ui.row().classes('ml-auto'):
            # 绑定按钮状态到 header_visible (双向绑定)
            ui.checkbox('显示 Header', value=True).bind_value(state, 'header_visible')
            ui.button('隐藏', on_click=lambda: header.hide()).props('flat')
            ui.button('显示', on_click=lambda: header.show()).props('flat')
            ui.button('切换', on_click=lambda: header.toggle()).props('flat')

        # 监听 header 的 value 变化事件
        header.on_value_change(lambda e: ui.notify(f'Header 状态变化: {e.value}'))

    [ui.label(f'页面内容第 {i} 行') for i in range(100)]

ui.run()

```

3. 样式定制：结合 Tailwind 类与 Quasar Props

```

from nicegui import ui

@ui.page('/styled_header')
def styled_header_demo():
    # 自定义高度、居中对齐、带阴影的 header
    with ui.header(
        elevated=True, # 阴影效果
        bordered=True, # 底部边框
        wrap=False, # 禁止内容换行
        add_scroll_padding=True # 自动添加滚动内边距
    ).style('height: 70px').classes('flex justify-center items-center'):
        # 左侧logo
        ui.label('LOGO').classes('text-2xl font-bold text-blue-600')
        # 中间导航菜单 (通过 ml-auto/mr-auto 实现居中)
        with ui.row().classes('mx-8'):
            ui.link('首页', '/').classes('px-4 py-2 text-gray-700 hover:text-blue-600')
            ui.link('产品', '/products').classes('px-4 py-2 text-gray-700 hover:text-blue-600')
            ui.link('关于我们', '/about').classes('px-4 py-2 text-gray-700 hover:text-blue-600')
        # 右侧搜索框

```

```
with ui.row().classes('m1-auto'):
    ui.input(placeholder='搜索...').props('rounded outlined').style('width: 200px')
    ui.button(icon='search').props('flat rounded-full m1-2')

[ui.label(f'页面内容第 {i} 行') for i in range(100)]

ui.run()
```

六、注意事项

- 固定定位与滚动内边距**：当 `fixed=True` 时，建议保留 `add_scroll_padding=True`，否则锚点链接的目标元素可能被 header 遮挡；若手动设置页面顶部内边距，可将 `add_scroll_padding` 设为 `False`。
- 内容换行控制**：`wrap=False` 时，header 内容超出宽度会被截断，需确保内容宽度不超过页面，或通过响应式类（如 `md:hidden`）隐藏多余内容。
- 样式优先级**：`style` 方法设置的内联样式优先级高于 `classes` 中的类样式，若需覆盖内联样式，可使用 `!important` 关键字（如 `classes('bg-white !important')`）。
- 数据绑定的严格模式**：绑定数据时，`strict` 参数（默认 `None`）会检查目标对象是否存在指定属性，非字典对象建议开启 `strict=True` 以避免错误。
- 事件绑定的节流控制**：使用 `on()` 方法绑定高频事件（如 `scroll`）时，可通过 `throttle` 参数设置节流时间（单位：秒），避免服务器压力过大。
- 版本兼容性**：`html_id` 属性需 NiceGUI 2.16.0+ 版本，`js_handler` 支持同时指定 Python 与 JS 处理器需 2.18.0+ 版本，使用时需注意版本匹配。

七、扩展参考

- 如需更复杂的 header 交互（如下拉菜单、响应式折叠），可结合 `ui.menu`、`ui.dropdown` 组件使用。
- Quasar Header 原生属性可通过 `props()` 方法传递，详细支持的 props 可参考 [Quasar Header 文档](#)。
- 样式定制可结合 Tailwind CSS 类（如 `flex`、`justify-between`、`items-center`）或 Quasar 类（如 `q-pa-md`、`q-mr-auto`）实现。

ui.header

在 NiceGUI 中，`ui.header` 是专门用于构建 ** 页面头部（页眉）** 的组件，它封装了头部布局的常用样式和交互逻辑，是实现导航栏、页面标题栏、操作栏等头部区域的核心组件。相较于手动用 `ui.row` 或 `ui.element` 构建头部，`ui.header` 提供了更贴合页面头部场景的默认样式和便捷配置，同时支持与其他组件（如 `ui.menu`、`ui.button`、`ui.input`）的无缝结合。

本文将从**基础用法**、**核心配置**、**样式定制**、**交互拓展**和**实战场景**五个维度，详细阐述 `ui.header` 的使用方法，帮助你快速构建美观且功能完整的页面头部。

一、ui.header 基础用法

`ui.header` 本质是一个**横向布局的容器**（底层基于 CSS Flexbox），默认具备以下特性：

- 水平排列子组件，默认**垂直居中**、**两端对齐**（可通过配置调整）；
- 自带合理的内边距和高度，适配页面头部的视觉规范；
- 支持嵌套其他 NiceGUI 组件，如按钮、菜单、输入框、徽标等。

1. 最简示例

创建一个基础的页面头部，包含标题和操作按钮：

```
from nicegui import ui

# 基础页面头部
with ui.header():
    # 左侧标题
    ui.label('NiceGUI 管理系统').classes('text-2xl font-bold')
    # 右侧操作按钮
    with ui.row().classes('gap-2'): # 按钮组横向排列，间距2
        ui.button('登录')
        ui.button('注册').classes('bg-primary text-white')

# 主内容区域（避免与头部重叠）
ui.label('页面主内容').classes('p-4')

ui.run()
```

2. 固定头部

页面头部通常需要**固定在页面顶部**（滚动页面时不随内容移动），通过 `classes` 或 `style` 可快速实现，这是 `ui.header` 最常用的场景之一：

```
from nicegui import ui

# 固定在顶部的头部（z-index 确保不被其他内容覆盖）
with ui.header().classes('fixed top-0 left-0 w-full z-10 bg-white shadow-md'):
    ui.label('固定头部示例').classes('text-xl font-bold')
    ui.button('菜单').classes('ml-auto') # ml-auto: 右对齐

# 主内容（添加顶部外边距，避免被头部遮挡）
ui.label('向下滚动查看效果').classes('p-4 mt-16')
# 长文本用于测试滚动
ui.markdown('# 滚动测试\n' * 20)

ui.run()
```

关键样式说明：

- `fixed top-0 left-0 w-full`：固定定位，占满屏幕宽度；
- `z-10`：设置层级，避免被其他组件覆盖；
- `bg-white shadow-md`：白色背景 + 轻微阴影，提升视觉层次；
- `mt-16`：主内容顶部外边距（适配头部高度）。

二、ui.header 核心配置

ui.header 支持通过方法链和Tailwind CSS 类配置布局、对齐方式、尺寸等核心属性，以下是最常用的配置项：

1. 对齐方式

ui.header 基于 Flexbox 布局，可通过 Tailwind 类调整子组件的**水平对齐**和**垂直对齐**：

对齐需求	对应的 Tailwind 类	说明
子组件左对齐	justify-start	默认左对齐
子组件居中对齐	justify-center	水平居中
子组件两端对齐	justify-between (默认)	左右元素分别贴边
子组件右对齐	justify-end	所有元素右对齐
子组件垂直置顶	items-start	垂直方向顶部对齐
子组件垂直居中	items-center (默认)	垂直方向居中 (常用)
子组件垂直置底	items-end	垂直方向底部对齐

示例：

```
from nicegui import ui

# 居中对齐的头部（适合单标题场景）
with ui.header().classes('justify-center bg-gray-100 p-4'):
    ui.label('居中标题的头部').classes('text-2xl font-bold')

# 两端对齐的头部（左侧logo，右侧操作栏）
with ui.header().classes('justify-between bg-blue-50 p-4 mt-2'):
    ui.label('LOGO').classes('text-xl font-bold text-blue-600')
    ui.input(placeholder='搜索...').classes('w-64')

ui.run()
```

2. 尺寸与内边距

通过 Tailwind 类调整头部的高度、内边距和宽度，适配不同的设计需求：

- 高度：h-16（高度 16，对应 4rem）、h-20 等；
- 内边距：px-4（水平内边距 4）、py-2（垂直内边距 2）、p-6（上下左右内边距 6）；
- 宽度：w-full（占满父容器，默认）、w-3/4（占 3/4 宽度）等。

示例：


```

from nicegui import ui

# 自定义高度和内边距的头部
with ui.header().classes('h-20 px-8 bg-gray-800 text-white items-center'):
    ui.label('自定义尺寸头部').classes('text-2xl')
    ui.button('设置').classes('ml-auto bg-gray-600 hover:bg-gray-500')

ui.run()

```

3. 子组件的间距与定位

在 `ui.header` 中，子组件的间距和定位是高频需求，常用技巧包括：

- `gap-x / gap-y`：设置子组件的间距（如 `gap-4` 表示间距 4）；
- `ml-auto / mr-auto`：将单个组件推到右侧 / 左侧（实现“左侧 logo + 右侧操作栏”的经典布局）；
- `ui.row() / ui.column()`：嵌套容器实现子组件的分组排列。

经典布局示例（左侧 logo + 中间导航 + 右侧操作）：

```

from nicegui import ui

with ui.header().classes('justify-between items-center bg-white shadow-sm p-4 w-full'):
    # 左侧LOGO
    ui.label('MY LOGO').classes('text-xl font-bold text-red-600')

    # 中间导航（通过 ml-auto/mr-auto 居中）
    with ui.row().classes('gap-6 ml-auto mr-auto'):
        ui.link('首页', '/').classes('text-lg hover:text-red-600')
        ui.link('产品', '/products').classes('text-lg hover:text-red-600')
        ui.link('文档', '/docs').classes('text-lg hover:text-red-600')

    # 右侧操作栏
    with ui.row().classes('gap-3'):
        ui.button('登录').classes('text-sm')
        ui.button('注册').classes('text-sm bg-red-600 text-white')

ui.run()

```

三、`ui.header` 样式定制

`ui.header` 支持默认样式覆盖和自定义 CSS，满足个性化的设计需求，主要有两种定制方式：

1. 通过 Tailwind CSS 类快速定制

NiceGUI 内置了 Tailwind CSS，可直接通过 `classes()` 方法为 `ui.header` 设置背景色、文字色、阴影、圆角等样式：

```

from nicegui import ui

# 个性化样式的头部
with ui.header().classes('''
    w-full h-18 bg-gradient-to-r from-purple-500 to-pink-500
    text-white shadow-lg rounded-b-lg items-center px-6
'''):
    ui.label('渐变背景头部').classes('text-2xl font-bold')
    ui.button('菜单').classes('ml-auto bg-white text-purple-600 hover:bg-gray-100')

ui.run()

```

样式说明:

- `bg-gradient-to-r from-purple-500 to-pink-500`: 从紫色到粉色的横向渐变背景;
- `shadow-lg`: 大阴影效果;
- `rounded-b-lg`: 底部圆角;
- `h-18`: 自定义高度 (Tailwind 支持任意高度值) 。

2. 通过自定义 CSS 深度定制

对于更复杂的样式需求 (如伪类、动画), 可通过 `ui.add_css()` 编写自定义 CSS, 为 `ui.header` 添加专属样式:

```

from nicegui import ui

# 添加自定义CSS
ui.add_css('''
    /* 自定义头部样式 */
    .custom-header {
        border-bottom: 3px solid #2563eb;
        transition: background-color 0.3s ease;
    }
    /* 鼠标悬停时改变背景 */
    .custom-header:hover {
        background-color: #f8fafc;
    }
    /* 子组件样式 */
    .custom-header .header-title {
        color: #2563eb;
        font-family: 'Arial', sans-serif;
    }
''')

# 使用自定义CSS类的头部
with ui.header().classes('custom-header p-4 w-full items-center'):
    ui.label('自定义CSS头部').classes('header-title text-2xl font-bold')
    ui.button('操作').classes('ml-auto bg-blue-600 text-white')

ui.run()

```

四、`ui.header` 交互拓展

`ui.header` 不仅是静态布局容器，还能结合 NiceGUI 的交互组件实现复杂的头部功能，如下拉菜单、搜索框、暗黑模式切换等。

1. 集成下拉菜单 (`ui.menu`)

实现头部的下拉菜单（如用户中心、设置菜单）：

```
from nicegui import ui

with ui.header().classes('fixed top-0 left-0 w-full bg-white shadow-md p-4 items-center z-10'):
    ui.label('系统管理').classes('text-xl font-bold')

    # 右侧用户菜单
    with ui.menu_button('用户中心', icon='person').classes('ml-auto'):
        ui.menu_item('个人资料', on_click=lambda: ui.notify('查看个人资料'))
        ui.menu_item('修改密码', on_click=lambda: ui.notify('修改密码'))
        ui.menu_separator()
        ui.menu_item('退出登录', on_click=lambda: ui.notify('退出成功'))

    # 主内容
    ui.label('主内容区域').classes('p-4 mt-16')

ui.run()
```

2. 集成搜索框 (`ui.input`)

实现头部的搜索功能，结合按钮触发搜索：

```
from nicegui import ui

with ui.header().classes('p-4 bg-gray-100 items-center'):
    ui.label('搜索示例').classes('text-xl font-bold')

    # 搜索框（居中）
    search_input = ui.input(placeholder='请输入搜索内容...').classes('w-96 ml-auto mr-auto')
    # 搜索按钮
    ui.button('搜索', icon='search').classes('ml-2 bg-blue-500 text-white').on_click(
        lambda: ui.notify(f'搜索: {search_input.value}')
    )

ui.run()
```

3. 集成暗黑模式切换

结合 NiceGUI 的 `ui.dark_mode` 实现头部的暗黑模式切换按钮：

```

from nicegui import ui

with ui.header().classes('fixed top-0 left-0 w-full bg-white dark:bg-gray-800 shadow-md p-4 items-center z-10'):
    ui.label('暗黑模式切换').classes('text-xl font-bold dark:text-white')
    # 暗黑模式切换开关
    ui.switch('暗黑模式').bind_value(ui.dark_mode, 'value').classes('ml-auto')

# 主内容
ui.label('切换模式查看效果').classes('p-4 mt-16 dark:text-white')

ui.run()

```

五、实战场景：完整的企业级头部布局

结合以上知识点，实现一个包含**LOGO**、**导航栏**、**搜索框**、**用户菜单**、**暗黑模式**的企业级页面头部，覆盖 `ui.header` 的核心使用场景：

```

from nicegui import ui

# 固定头部，适配暗黑模式
with ui.header().classes(''
    fixed top-0 left-0 w-full z-10
    bg-white dark:bg-gray-800 shadow-md
    p-4 items-center justify-between
''):
    # 左侧LOGO区域
    with ui.row().classes('items-center gap-2'):
        ui.icon('dashboard').classes('text-2xl text-blue-600 dark:text-blue-400')
        ui.label('企业管理平台').classes('text-xl font-bold dark:text-white')

    # 中间导航栏
    with ui.row().classes('gap-6 hidden md:flex'): # 平板及以上显示
        for item in ['首页', '数据报表', '用户管理', '系统设置']:
            ui.link(item, f'/{item}').classes('text-gray-700 dark:text-gray-200 hover:text-blue-600 dark:hover:text-blue-400')

    # 右侧功能区
    with ui.row().classes('items-center gap-4'):
        # 搜索框（小屏幕隐藏）
        search_input = ui.input(placeholder='搜索...').classes('w-64 hidden lg:block dark:bg-gray-700 dark:text-white')
        ui.button(icon='search').classes('lg:hidden dark:bg-gray-700').on_click(
            lambda: ui.notify(f'搜索: {search_input.value}') if search_input.value else
            ui.notify('请输入搜索内容')
        )
        # 暗黑模式切换
        ui.switch('').bind_value(ui.dark_mode, 'value').classes('dark:text-white')
        # 用户下拉菜单
        with ui.menu_button(avatar='https://picsum.photos/id/1005/40/40',
            icon='person').classes('dark:bg-gray-700'):
            ui.menu_item('个人资料', on_click=lambda: ui.notify('查看个人资料'))

```

```
ui.menu_item('消息中心', on_click=lambda: ui.notify('查看消息'))
ui.menu_separator()
ui.menu_item('退出登录', on_click=lambda: ui.notify('退出成功', type='success'))

# 主内容区域（适配头部高度和暗黑模式）
with ui.column().classes('p-6 mt-20 dark:bg-gray-900 dark:text-white min-h-screen'):
    ui.label('欢迎使用企业管理平台').classes('text-3xl font-bold mb-4')
    ui.markdown('这是一个基于 NiceGUI `ui.header` 实现的企业级头部布局，支持：\n- 响应式设计\n- 暗黑模式\n- 下拉菜单\n- 搜索功能\n- 导航链接')

ui.run(title='企业级头部布局示例', dark_mode_preference='system')
```

核心特性说明：

- 响应式设计：**导航栏和搜索框在小屏幕上隐藏，通过图标按钮替代；
- 暗黑模式适配：**所有组件均添加了 `dark:` 前缀的 Tailwind 类，支持暗黑模式切换；
- 交互完整：**包含搜索、导航、用户菜单、暗黑模式等企业级功能；
- 视觉层次：**通过颜色、间距、阴影提升头部的视觉体验。

六、总结

`ui.header` 是 NiceGUI 中构建页面头部的**专用组件**，其核心价值在于：

- 简化布局：**提供贴合头部场景的默认 Flexbox 布局，无需手动编写基础样式；
- 灵活拓展：**支持嵌套所有 NiceGUI 组件，实现从简单标题到复杂功能栏的各类头部；
- 适配性强：**结合 Tailwind CSS 可快速实现响应式设计和暗黑模式适配；
- 交互友好：**与 `ui.menu`、`ui.link`、`ui.switch` 等组件无缝集成，轻松实现交互功能。

在实际开发中，`ui.header` 通常与 `ui.sidebar`（侧边栏）、`ui.footer`（页脚）配合使用，构成完整的页面布局体系。掌握 `ui.header` 的使用，能让你在 NiceGUI 中快速构建专业、美观的页面头部，提升整体项目的开发效率和用户体验。